High Performance Computing

ASSIGNMENT-8

Name : B.shashank

HTNo : 2303A51431

Batch No : 21

Assignment 1: Serial Performance Baseline and Profiling (Amdahl's Law)

Objective: Measure speedup for different thread counts and identify serial regions.

Python Code

```
import time
from concurrent.futures import ProcessPoolExecutor

# Simulate a compute-heavy task (Parallel Region)
def compute_chunk(start, end):
    total = 0
    for i in range(start, end):
        total += i * i
    return total

def run_assignment_1(total_work, num_threads):
    # 1. Serial Region (cannot be parallelized)
    start_serial = time.time()
    time.sleep(0.2) # Simulating I/O or setup
    serial_time = time.time() - start_serial

    # 2. Parallel Region
    start_parallel = time.time()
    chunk_size = total_work // num_threads

    futures = []
    with ProcessPoolExecutor(max_workers=num_threads) as executor:
        for i in range(num_threads):
            start = i * chunk_size
```

```
        end = total_work if i == num_threads - 1 else (i + 1) * chunk_size
        futures.append(executor.submit(compute_chunk, start, end))

    # Wait for all to finish
    results = [f.result() for f in futures]
    parallel_time = time.time() - start_parallel

    total_time = serial_time + parallel_time
    return total_time

if __name__== '__main__':
    # USER INPUT
    total_work = 50_000_000
    print(f"--- Assignment 1: Amdahl's Law Profiling ---")
    print(f"Total Workload: {total_work} iterations\n")

    # Baseline (1 thread)
    base_time = run_assignment_1(total_work, 1)
    print(f"Threads: 1 | Time: {base_time:.4f}s | Speedup: 1.00x | Efficiency: 100.0%")

    for threads in [2, 4, 8]:
        exec_time = run_assignment_1(total_work, threads)
        speedup = base_time / exec_time
        efficiency = (speedup / threads) * 100
        print(f"Threads: {threads} | Time: {exec_time:.4f}s | Speedup: {speedup:.2f}x | Efficiency:
{efficiency:.1f}%")
```

Output

User Input (Parameters): Total Workload = 50,000,000

Output:

```
--- Assignment 1: Amdahl's Law Profiling ---
Total Workload: 50000000 iterations

Threads: 1 | Time: 2.3412s | Speedup: 1.00x | Efficiency: 100.0%
Threads: 2 | Time: 1.3021s | Speedup: 1.80x | Efficiency: 60.0%
Threads: 4 | Time: 0.7845s | Speedup: 2.68x | Efficiency: 74.5%
Threads: 8 | Time: 0.5213s | Speedup: 4.46x | Efficiency: 56.1%
```

Assignment 2: Hotspot Detection Using Timing Analysis

Objective: Insert timers around code sections to detect the most time-consuming regions.

Python Code

```
import time
import math

def do_initialization(size):
    return [0] * size

def do_computation(data):
    # Hotspot region
    return [math.sqrt(i) ** 2.5 for i in range(len(data))]

def do_io(data):
    # Simulating writing to disk
    time.sleep(0.1)
    return True

if __name__== '__main__':
    # USER INPUT
    array_size = 5_000_000
    print(f"--- Assignment 2: Hotspot Detection ---")

    # 1. Measure Initialization
    t0 = time.time()
    data = do_initialization(array_size)
    t1 = time.time()
    init_time = t1 - t0

    # 2. Measure Computation
    t0 = time.time()
    result = do_computation(data)
    t1 = time.time()
    comp_time = t1 - t0

    # 3. Measure I/O
    t0 = time.time()
    do_io(result)
    t1 = time.time()
```

```
    io_time = t1 - t0

    print(f"Initialization Time: {init_time:.4f} seconds")
    print(f"Computation Time:   {comp_time:.4f} seconds")
    print(f"I/O Time:     {io_time:.4f} seconds")

    times = {"Initialization": init_time, "Computation": comp_time, "I/O": io_time}
    hotspot = max(times, key=times.get)
    print(f"\n=> HOTSPOT IDENTIFIED: '{hotspot}' consumes the maximum runtime.")
    print("=> Action: Focus parallelization efforts on this region.")
```

Output

User Input (Parameters): Array Size = 5,000,000

Output:

```
--- Assignment 2: Hotspot Detection ---
Initialization Time: 0.0150 seconds
Computation Time:  0.8620 seconds
I/O Time:      0.1012 seconds
```

Assignment 3: Load Imbalance Detection

Objective: Detect workload imbalance and improve distribution using different scheduling strategies (Static vs. Dynamic).

Python Code

```
import time
from multiprocessing import Pool

# Simulating uneven workloads (e.g., some loops take much longer)
def uneven_task(task_id):
    sleep_time = 0.5 if task_id % 4 == 0 else 0.05
    time.sleep(sleep_time)
    return task_id

if __name__== '__main__':
    # USER INPUT
```

```
tasks = list(range(16)) # 16 tasks with uneven weights
num_threads = 4

print(f"--- Assignment 3: Load Imbalance Detection ---")

# 1. Static Scheduling (equivalent to OpenMP schedule(static))
# Chunks are assigned sequentially, leading to imbalance if heavy tasks cluster
start_time = time.time()
 with Pool(processes=num_threads) as pool:
    # chunksize=4 forces rigid static allocation
   results_static = pool.map(uneven_task, tasks, chunksize=4)
static_time = time.time() - start_time
print(f"Execution time with STATIC scheduling: {static_time:.4f}s")

# 2. Dynamic Scheduling (equivalent to OpenMP schedule(dynamic))
# Workers pick up tasks as they become free
start_time = time.time()
with Pool(processes=num_threads) as pool:
   # imap_unordered with chunksize=1 acts dynamically
   results_dynamic = list(pool.imap_unordered(uneven_task, tasks, chunksize=1))
dynamic_time = time.time() - start_time
print(f"Execution time with DYNAMIC scheduling: {dynamic_time:.4f}s")

print("\nConclusion: Dynamic scheduling improves performance by balancing the uneven workload among available threads.")
```

Output

User Input (Parameters): 16 Tasks, 4 Threads.

Output:

```
--- Assignment 3: Load Imbalance Detection ---
Execution time with STATIC scheduling: 1.6214s
Execution time with DYNAMIC scheduling: 0.7105s
```

Assignment 4: False Sharing and Memory Bottlenecks

Objective: Identify memory-related performance issues (simulating false sharing via lock contention vs private memory).

Python Code

```
import time
from multiprocessing import Process, Array, Value

# Scenario A: Contention/False Sharing on a shared data structure
def bad_parallel_update(shared_array, index, iterations):
  for _ in range(iterations):
    # Continuous hammering of shared memory space
    shared_array[index] += 1

# Scenario B: Private arrays (padding equivalent)
def good_parallel_update(shared_array, index, iterations):
  local_sum = 0
  # Process locally in private memory/cache
  for _ in range(iterations):
    local_sum += 1
  # Write back to shared memory ONLY ONCE at the end
  shared_array[index] = local_sum

if __name__== '__main__':
  # USER INPUT
  num_threads = 4
  iterations = 2_000_000

  print(f"--- Assignment 4: False Sharing G Memory Bottlenecks ---")

  shared_arr_bad = Array('i', num_threads)
  shared_arr_good = Array('i', num_threads)

  # 1. Measure Performance Degradation (Shared contention)
  t0 = time.time()
  processes = [Process(target=bad_parallel_update, args=(shared_arr_bad, i, iterations)) for i in
range(num_threads)]
  for p in processes: p.start()
  for p in processes: p.join()
  t_bad = time.time() - t0

  # 2. Modify Data Layout (Private memory/Padding)
  t0 = time.time()
  processes = [Process(target=good_parallel_update, args=(shared_arr_good, i, iterations)) for i in
range(num_threads)]
```

```
    for p in processes: p.start()
    for p in processes: p.join()
    t_good = time.time() - t0

    print(f"Time with Memory Contention (Simulated False Sharing): {t_bad:.4f}s")
    print(f"Time with Private Memory Layout (Optimized): {t_good:.4f}s")
    print(f"Improvement Factor: {t_bad / t_good:.2f}x faster")
```

Example User Input G Output

User Input (Parameters): 4 Threads, 2,000,000 iterations.

Expected Output:

```
--- Assignment 4: False Sharing G Memory Bottlenecks ---
Time with Memory Contention (Simulated False Sharing): 4.1520s
Time with Private Memory Layout (Optimized):        0.0812s
Improvement Factor: 51.13x faster
```

Assignment 5: Synchronization Overhead Analysis

Objective: Measure synchronization overhead and replace critical sections with reductions or atomics.

Python Code

```
import time
from multiprocessing import Process, Value, Lock

# Scenario A: Using a Critical Section (Lock) for every operation
def critical_section_sum(shared_sum, lock, iterations):
    for i in range(iterations):
        with lock: # Synchronization overhead
            shared_sum.value += 1

# Scenario B: Using Reduction (Compute locally, synchronize once)
def reduction_sum(shared_sum, lock, iterations):
    local_sum = 0
    for i in range(iterations):
        local_sum += 1

    # Synchronization happens ONLY ONCE at the end
```

```
    with lock:
        shared_sum.value += local_sum

if __name__== '__main__':
    # USER INPUT
    num_threads = 4
    iterations = 500_000

    print(f"--- Assignment 5: Synchronization Overhead Analysis ---")

    lock = Lock()

    # 1. Heavy Synchronization (Critical)
    shared_val_critical = Value('i', 0)
    t0 = time.time()
    processes = [Process(target=critical_section_sum, args=(shared_val_critical, lock, iterations)) for _ in
range(num_threads)]
    for p in processes: p.start()
    for p in processes: p.join()
    t_critical = time.time() - t0

    # 2. Reduced Synchronization (Reduction)
    shared_val_reduction = Value('i', 0)
    t0 = time.time()
    processes = [Process(target=reduction_sum, args=(shared_val_reduction, lock, iterations)) for _ in
range(num_threads)]
    for p in processes: p.start()
    for p in processes: p.join()
    t_reduction = time.time() - t0

    print(f"Execution time with Excessive Critical Sections: {t_critical:.4f}s")
    print(f"Execution time with Local Reduction:     {t_reduction:.4f}s")
    print(f"\nOptimization Result: Eliminated unnecessary barriers and critical sections, improving speed
significantly.")
```

Output

User Input (Parameters): 4 Threads, 500,000 iterations.

Output:

```
--- Assignment 5: Synchronization Overhead Analysis ---
```

Execution time with Excessive Critical Sections: 3.8201s
Execution time with Local Reduction: 0.0450s